

Assignment 6: CART

AUTHOR

Talha Gulzar

PUBLISHED

October 1, 2024

Context

Let's take a look at the data located in ToyotaCorolla.csv. Each row represents a used car, with the collection of columns ranging from a fairly detailed model name (Model) to performance characteristics like horsepower (HP), and engine size (cc). Our goal here is to predict the numeric target Price. You can imagine several different applications for this sort of algorithm. Perhaps work for an insurance company, and are dissatisfied with the industry standard car value estimates, or perhaps you want to make money at auto auctions by bidding on cars that you think are currently significantly undervalued.

The first model that pops to mind for this type of problem is likely linear regression, and that's not bad intuition. After all, linear regression is a simple and often powerful technique that has the potential to tell us a lot about our problem. At the same time, it, by its very nature, forces us into a very particular worldview – namely that the target and every input are related linearly. This is a bold claim, and one that is often simply not true. Trees, on the other hand, try to uncover the relationship between the most important inputs and the target, whatever form they might take.

Question 1

To get ready for modeling, complete the following data management steps:

- Import the data and call it "df".
- Remove the variable "Model." This should have no predictive power given the subtle differences between the same brand of car.
- Convert variables to factors, if necessary. Note that predictors in CART can be either categorical or numeric.
- Partition the data using a 60-40 training-test split using random seed 1234.

```
#insert code
df <- read.csv("ToyotaCorolla.csv")
df$Model = NULL
df$Fuel_Type = as.factor(df$Fuel_Type)
df$Met_Color = as.factor(df$Met_Color)
df$Automatic = as.factor(df$Automatic)

set.seed(1234)
N <- nrow(df)
trainingSize <- round(N*0.6)
trainingCases <- sample(N, trainingSize)
```

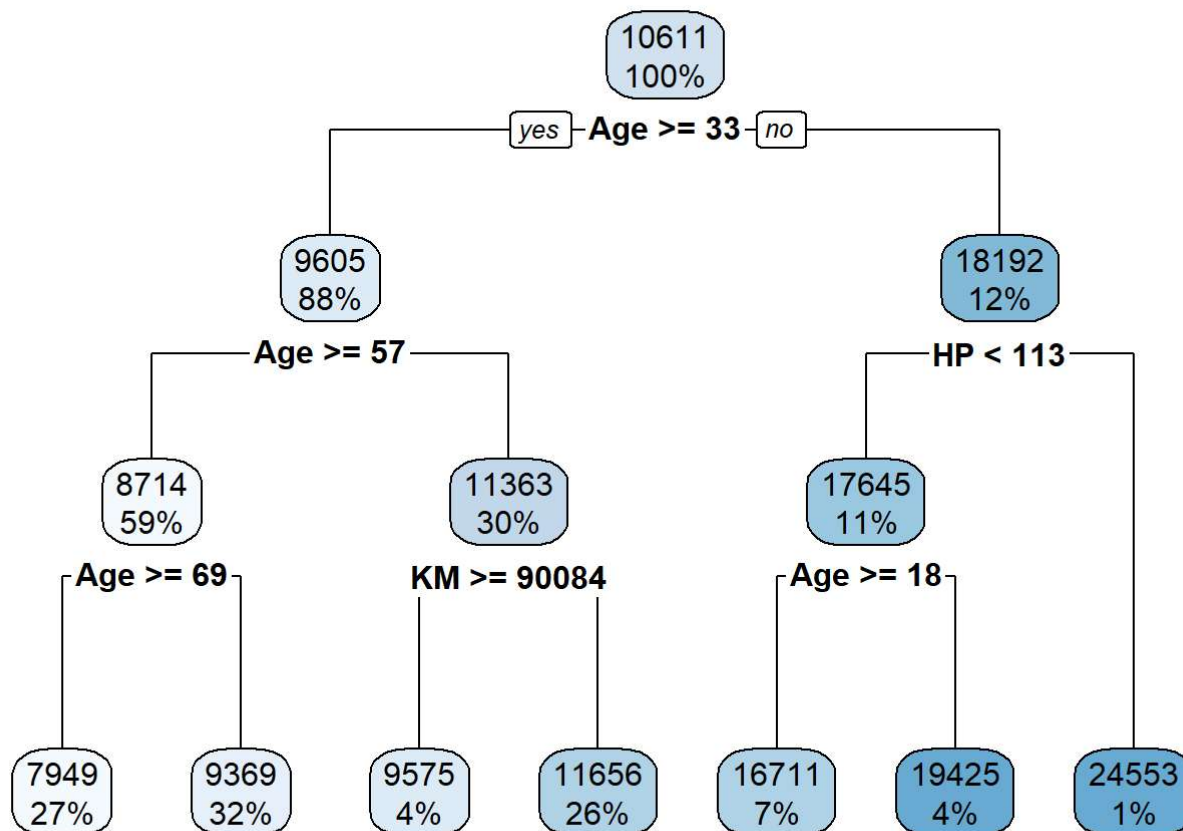
```
training <- df[trainingCases,]
test <- df[-trainingCases,]
```

Question 2

Build your CART model using all available predictors to predict Price. Use default stopping rules. Also ask R to display the resulting tree using `rpart.plot`. When doing so, add `digits=-2` as an option/parameter within your `rpart.plot()` function if you wish to remove scientific notation, such as `rpart.plot(model, digits=-2)`.

```
#insert code
library(rpart)
library(rpart.plot)

model <- rpart(Price ~ ., data=training)
rpart.plot(model, digits= -2)
```



Question 3

In class, each node contained three numbers. In your tree for Question 2, you should only see two numbers in each node. What is missing and why do you think that is?

Answer: In class we experienced a classification model instead of a regression model. The model explained the probabilities of 0s and 1s where as in the above outcome the probability of a numerical outcome is explained hence only two numbers are shown in each node.

Question 4

In examining your tree from Question 2, if a car is 72 months old and has been driven 102,000 km, what is the predicted price? Note that the Age variable is in months and KM variable is in kilometers. Price is in Euros.

Answer: 7949 Euro

Question 5

What would the price of a 50-month-old car with 20 thousand kilometers on the odometer be according to your regression tree with default stopping rules? Price is in Euros.

Answer: 11656 Euro

Question 6

Calculate the MAPE and RMSE for your model. Interpret each of them in context. Answer: A MAPE of 0.1066997 (or 10.67%) means that, on average, the model's predicted car prices deviate from the actual prices by about 10.67%. An RMSE of 1496.334 means that, on average, the difference between the predicted and actual car prices is about 1,496.33 euros

```
predictions <- predict(model, test)
observations <- test$Price

# Get the errors between observed and predictions (so these are "residuals")
errors <- observations - predictions

#Calculate the MAPE (Mean Absolute Percentage Error) for the model:
mape <- mean(abs(observations-predictions)/observations)
#View the MAPE
mape
```

```
[1] 0.1066997
```

```
#Calculate the Root Mean Square Error (RMSE)
rmse <- sqrt(mean((observations-predictions)^2))
#View the RMSE
rmse
```

```
[1] 1496.334
```

Question 7

What is the benchmark MAPE and RMSE associated with this model? Is your model useful?

Answer: Our regression tree model is better than the benchmark model, as it achieves both a lower RMSE (smaller average error in euros) and a lower MAPE (smaller percentage error), making it more accurate overall.

```
errors_bench <- observations - mean(training$Price)

# What is the mape compared to benchmark
mape_bench <- mean(abs(errors_bench)/observations)
mape_bench
```

```
[1] 0.2539698
```

```
rmse_bench <- sqrt(mean(errors_bench^2))
rmse_bench
```

```
[1] 3894.077
```

Question 8

You ran a model with the following parameters:

- minsplit=50,minbucket=20, cp=0.05

Your friend ran a model with the following parameters:

- minsplit=2,minbucket=1, cp=0.001

Do not run the models right now. Which model is more likely to be overfit, and why?

Answer: Because of its looser parameters—a lower minsplit=2, which supports splits with as few as two observations, and a minbucket=1, which permits terminal nodes with only one observation, my friend's model is more likely to overfit. With these parameters, a very intricate and complicated tree that closely matches the training set is produced, although it might not generalize well. Furthermore, numerous little splits are possible with the lesser complexity value (cp=0.001), which raises the possibility of overfitting even more. By contrast, a simpler, more generalizable tree is produced by my model's tougher settings (minsplit=50, minbucket=20, cp=0.05).

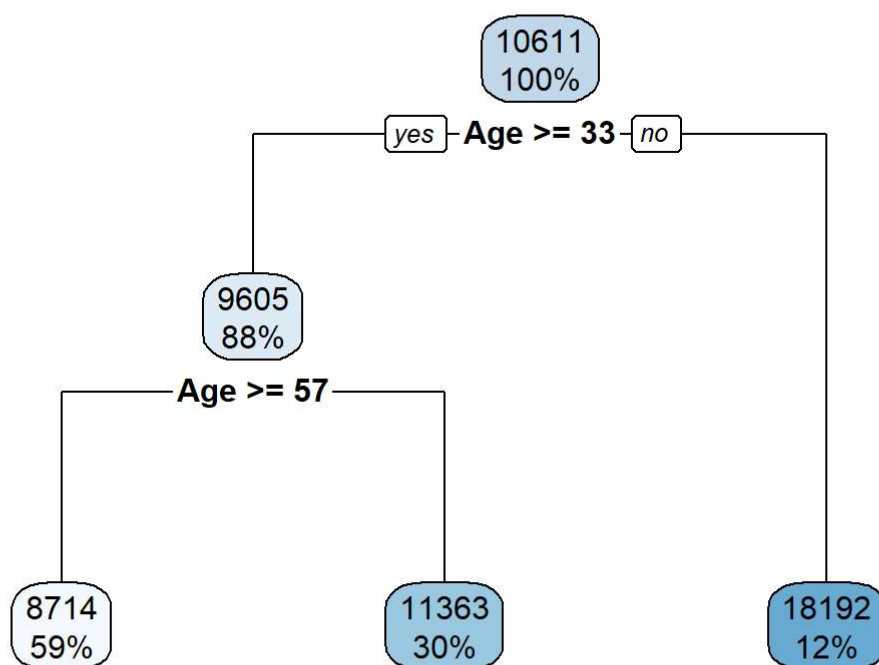
Question 9

Now run the two models from Question 8 using those parameters and calculate the MAPE for both models. Do the MAPE align with your hypothesis on which model is more likely to be overfit? Why?

Answer: Because my friend's model is more sophisticated and has fewer splitting restrictions (`minsplit=2`, `minbucket=1`, `cp=0.001`), it probably has a lower MAPE. As a result, even minute variations in the training set can be captured by the model, enabling it to fit the data quite closely. Consequently, it attains superior accuracy on the training set, which is demonstrated by a reduced MAPE.

The model is fitting noise and particular patterns in the training data, which may not translate well to new, unknown data, which could be the cause of the lower MAPE. It may perform poorly on test or validation data even while it performs better on the training set. So still regardless, my model is much better than my friend's model.

```
# Set stopping rules for splitting and rerun model
stoppingRules <- rpart.control(minsplit=50,minbucket=20, cp=0.05)
# These are the default stopping rules for rpart()
model2 <- rpart(Price ~ ., data=training, control=stoppingRules)
rpart.plot(model2, digits=-2)
```



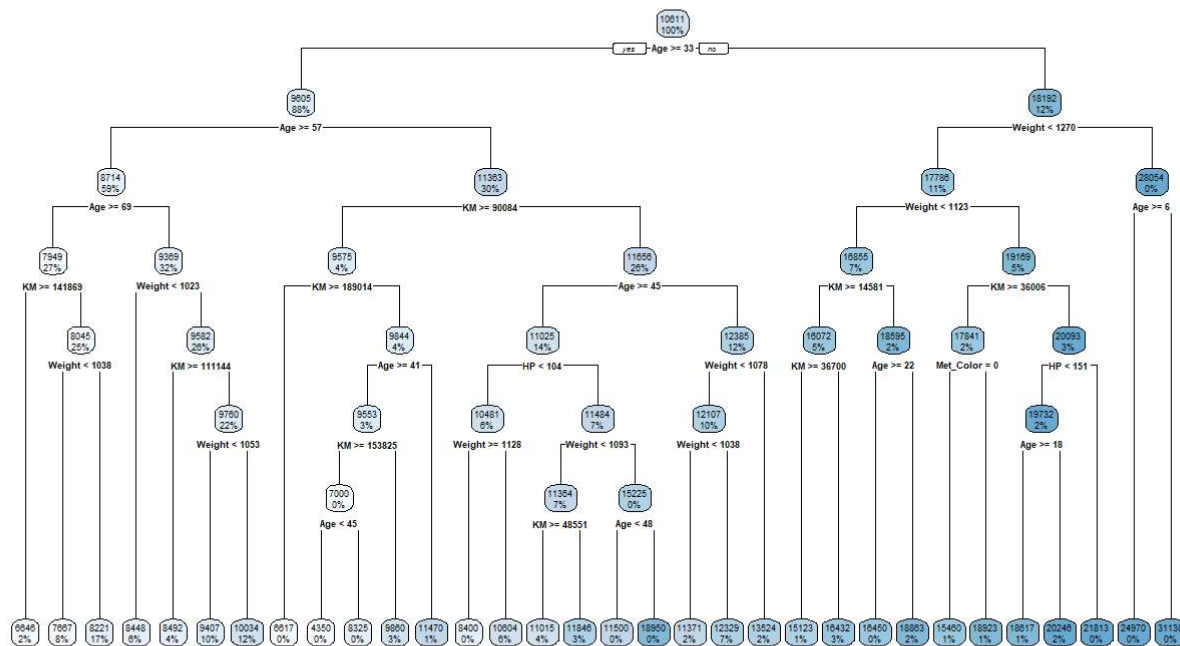
```
# Store the predictions and observed values of Price
predictions2 <- predict(model2, test)
observations <- test$Price

# Get the errors between observed and predictions (so these are "residuals")
errors <- observations - predictions2
```

```
#Calculate the MAPE (Mean Absolute Percentage Error) for the model:
mape1 <- mean(abs(observations-predictions)/observations)
#View the MAPE
mape1
```

[1] 0.1066997

```
# Set stopping rules for splitting and rerun model
stoppingRules <- rpart.control(minsplit=2,minbucket=1, cp=0.001)
# These are the default stopping rules for rpart()
model <- rpart(Price ~ ., data=training, control=stoppingRules)
rpart.plot(model, digits=-2)
```



```
# Store the predictions and observed values of Price
predictions <- predict(model, test)
observations <- test$Price

# Get the errors between observed and predictions (so these are "residuals")
errors <- observations - predictions

#Calculate the MAPE (Mean Absolute Percentage Error) for the model:
mape <- mean(abs(observations-predictions)/observations)
```

```
#View the MAPE  
mape
```

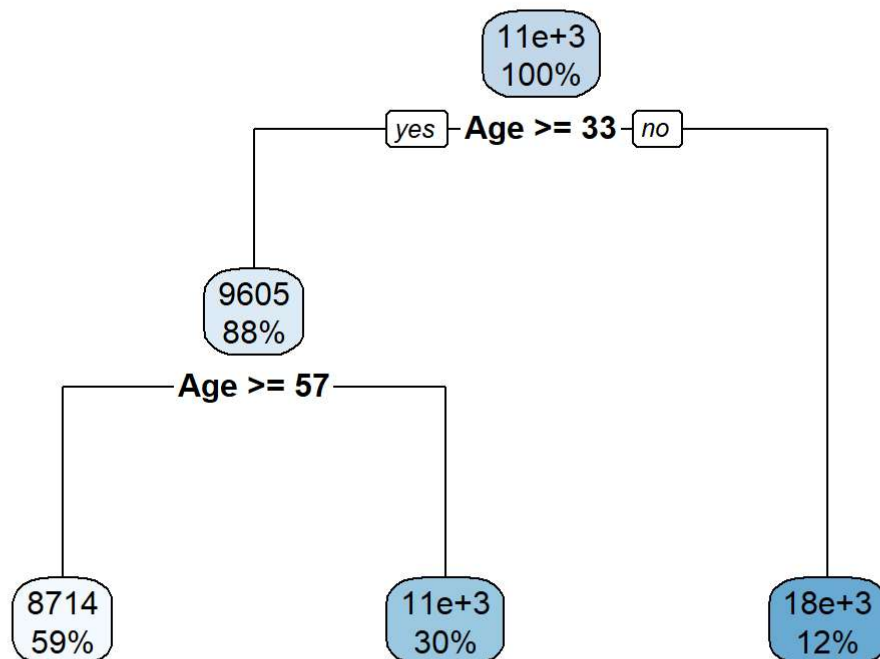
```
[1] 0.09936682
```

Question 10

Now, try to take your “overfitted” model and prune it. Calculate the MAPE for this pruned model and compare the MAPE to the MAPE of the “overfitted” model. What do you find? What might this say about our “overfitted” model?

Answer:

```
source("BabsonAnalytics.R")  
  
pruned = easyPrune(model12)  
rpart.plot(pruned)  
  
rpart.plot(model12)
```



```
predictions3 = predict(pruned,test)
observations =test$Price

mape3=mean(abs(observations-predictions3)/observations)

mape3
```

```
[1] 0.1302813
```

```
mape
```

```
[1] 0.09936682
```

Question 11

A friend of yours used a different random seed for their partition and has some confusing results. Your friend computed the MAPE for the default model and then pruned the model, finding that the two MAPE numbers matched almost exactly. Your friend has plotted both trees and confirmed that the the pruning is actually doing something, i.e., branches are actually being removed from the tree.

Can this be possible? Or is your friend making a mistake somewhere in the MAPE computation? Make a concrete argument one way or the other. You may cite specific properties of model performance, overfitting properties, or something else in order to make your case.

Answer: A small difference in MAPE between the overfitted and pruned models suggests that the overfitted model has made accurate predictions, and only some branches need pruning to improve the model. This could indicate that the dataset is relatively simple and that the splits made by the overfitted model are efficient. Pruning is based on cross-validation, where branches that introduce more error than the cross-validation average are removed. If fewer branches are pruned, it means the overfitted model's splits are close to the cross-validation error threshold. However, even with minimal pruning, it's important to evaluate the pruned model on test data to ensure it generalizes well and isn't simply fitting noise in the training data.